Python Lists & Tuples Master Test

30 Multiple Choice Questions (Theoretical Concepts & Coding Problems)

Subject: Python Programming **Topics:** Lists & Tuples **Total Questions:** 30 **Format:** MCQ with Answer Key

Q1. Which of the following correctly creates a single-element tuple in Python?

A) `my_tuple = (5)`

B) `my_tuple = (5,)`

C) `my_tuple = tuple[5]`

D) `my_tuple = 5`

Q2. What is the primary difference between a Python `list` and a `tuple`?

A) Lists store heterogeneous data, tuples store integers.

B) Lists are mutable, whereas tuples are immutable.

C) Tuples are faster to create, lists use less memory.

D) Lists cannot be nested, while tuples can.

Q3. What will be the output of the following code?

```
x = [1, 2, 3]
y = x
y.append(4)
print(x)
```

A) `[1, 2, 3]`

B) `[1, 2, 3, 4]`

C) `Error: Lists cannot be assigned this way`

D) `[4, 1, 2, 3]`

Q4. Which method is used to add an element to the *end* of a list?

A) `insert()`

B) `add()`

C) `append()`

D) `extend()`

Q5. What will be the output of `print(type((10)))`?

A) `<class 'tuple'>`

B) `<class 'list'>`

C) `<class 'int'>`

D) `<class 'dict'>`

Q6. What will the following code output?

```
numbers = [10, 20, 30, 40, 50]
print(numbers[-2])
```

A) `20`

B) `30`

C) `40`

D) `50`

Q7. How does `extend()` differ from `append()` when applied to a list?

A) `extend()` adds at the beginning, `append()` at end.

B) `extend()` unpacks iterable elements; `append()` adds item as a single element.

C) `extend()` returns a new list; `append()` modifies in place.

D) There is no difference between them.

Q8. What will be the output of this code snippet?

```
matrix = [[1, 2], [3, 4]] * 2
print(len(matrix))
```

A) `2`

B) `4`

C) `6`

D) `Error`

Q9. Which function returns a sorted *new* list without modifying the original list?

A) `list.sort()`

B) `sorted()`

C) `list.sorted()`

D) `order()`

Q10. What will be the output of the following slicing operation?

```
s = [1, 2, 3, 4, 5, 6]
print(s[1:5:2])
```

A) `[2, 3, 4, 5]`

B) `[2, 4]`

C) `[1, 3, 5]`

D) `[5, 3, 1]`

Q11. Can a tuple contain a mutable object, such as a list?

A) No, tuples can only contain immutable objects.

B) Yes, and elements inside the inner list can be modified.

C) Yes, but the inner list becomes immutable.

D) No, Python throws a `TypeError`.

Q12. What does the `pop()` method do by default when called on a list without arguments?

A) Removes and returns the first element.

B) Removes and returns a random element.

C) Removes and returns the last element.

D) Deletes the entire list from memory.

Q13. What will be the output of this code?

```
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b
print(c)
```

A) `[5, 7, 9]`

B) `[[1, 2, 3], [4, 5, 6]]`

C) `[1, 2, 3, 4, 5, 6]`

D) `Error`

Q14. Which built-in function can be used to pack multiple parallel sequences into tuples?

A) `pack()`

B) `combine()`

C) `zip()`

D) `map()`

Q15. What is the time complexity of looking up an item in a list using the `in` operator?

A) O(1) - Constant time

B) O(log n) - Logarithmic time

C) O(n) - Linear time

D) $O(n^2)$ - Quadratic time

Q16. What will be the output of this snippet?

```
tup = (1, 2, [3, 4])
tup[2].append(5)
print(tup)
```

A) `(1, 2, [3, 4])`

B) `(1, 2, [3, 4, 5])`

C) `TypeError: 'tuple' object does not support item assignment`

D) `(1, 2, 3, 4, 5)`

Q17. How do you find the number of occurrences of a specific element inside a tuple?

A) `tup.count(item)`

B) `tup.find(item)`

C) `count(tup, item)`

D) `tup.index(item)`

Q18. What will be the output of the following code?

```
nums = [10, 20, 30]
nums.insert(1, 15)
print(nums)
```

A) `[10, 20, 15, 30]`

B) `[15, 10, 20, 30]`

C) `[10, 15, 20, 30]`

D) `[10, 20, 30, 15]`

Q19. What is tuple unpacking?

A) Converting a tuple into a mutable list.

B) Extracting tuple elements into multiple variables in one statement.

C) Deleting a tuple from memory.

D) Unzipping a zipped file using tuples.

Q20. What will be printed by the following code?

```
def add_item(item, list_name=[]):
    list_name.append(item)
    return list_name

print(add_item(1))
print(add_item(2))
```

A) `[1]` followed by `[2]`

B) `[1]` followed by `[1, 2]`

C) `Error: mutable default argument`

D) `[1]` followed by `[2, 1]`

Q21. Which of the following operations on a tuple will raise a `TypeError`?

A) `tup[0]`

B) `tup + (4, 5)`

C) `tup[1] = 99`

D) `len(tup)`

Q22. What does slice assignment do in the following code?

```
lst = [1, 2, 3, 4]
lst[1:3] = [9, 9, 9]
print(lst)
```

A) Output: `[1, 9, 9, 9, 4]`

B) Raises a `ValueError` due to size mismatch.

C) Inserts `[9, 9, 9]` as a nested list inside `lst`.

D) Replaces the entire list with `[9, 9, 9]`.

Q23. Why are tuples generally preferred over lists for dictionary keys or set elements?

A) Tuples consume more memory than lists.

B) Tuples are immutable and hashable; lists are mutable and unhashable.

C) Tuples allow duplicate keys, which lists do not.

D) Tuples execute faster arithmetic calculations.

Q24. What will be the output of this code?

```
a = [1, 2, 3]
b = a[:]
b.append(4)
print(a)
```

A) `[1, 2, 3]`

B) `[1, 2, 3, 4]`

C) `Error`

D) `[4, 1, 2, 3]`

Q25. What is the purpose of the `remove()` method on lists?

A) Removes an element by its numerical index.

B) Removes the first occurrence of a specified value.

C) Removes and returns the last element of the list.

D) Clears all elements from the list.

Q26. What will be the output of the following code?

```
t = (1, 2, 3, 4, 5)
print(t[::-1])
```

A) `(1, 2, 3, 4, 5)`

B) `(5, 4, 3, 2, 1)`

C) `Error: Negative step not supported for tuples`

D) `[5, 4, 3, 2, 1]`

Q27. What does the `clear()` method do to a list?

A) Deletes the list variable entirely from the namespace.

B) Removes all items from the list, leaving it empty `[]`.

C) Resets all items to `None`.

D) Reverses the list items.

Q28. What will be the output of this code snippet using the star (`*`) unpacking operator?

```
a, *b, c = [1, 2, 3, 4, 5]
print(b)
```

A) `[2, 3, 4]`

B) `2`

C) `[2, 3, 4, 5]`

D) `[1, 2, 3, 4]`

Q29. Which underlying data structure layout do Python lists most closely resemble conceptually?

A) Singly linked lists

B) Dynamic arrays

C) Hash tables

D) Binary search trees

Q30. What will be the result of executing `tuple([1, 2, 3])`?

A) `[1, 2, 3]`

B) `(1, 2, 3)`

C) `Error: cannot cast list to tuple`

D) `1, 2, 3`

Answer Key & Detailed Explanations

Q1. Correct Answer: B

A single-element tuple requires a trailing comma, e.g., `(5,)`. Without the comma, `(5)` is treated merely as an integer enclosed in arithmetic grouping parentheses.

Q2. Correct Answer: B

Lists are mutable (their elements can be modified, added, or deleted after creation), whereas tuples are immutable (once created, their size and element bindings cannot be changed).

Q3. Correct Answer: B

`y = x` creates a shallow alias referencing the exact same list object in memory. Modifying `y` via `append(4)` directly mutates `x` as well.

Q4. Correct Answer: C

The `append()` method inserts a single element at the end of the list.

Q5. Correct Answer: C

Because `(10)` lacks a trailing comma, Python treats parentheses as math operators and evaluates the expression to the integer `10`, making its type `<class 'int'>`.

Q6. Correct Answer: C

Negative indices count backward from the end of the list. Index `-1` corresponds to `50`, and `-2` corresponds to `40`.

Q7. Correct Answer: B

`extend()` iterates over its argument and appends each element individually, whereas `append()` adds the argument as a single element (e.g., nesting a list inside another).

Q8. Correct Answer: B

The list `matrix` starts with 2 inner sublist elements. Multiplying a list by `2` duplicates its outer elements, resulting in a total length of `4`.

Q9. Correct Answer: B

The built-in `sorted()` function accepts any iterable and returns a brand-new sorted list, leaving the original list unaltered. `list.sort()` modifies the list in place and returns `None`.

Q10. Correct Answer: B

Slice syntax is `[start:stop:step]`. `s[1:5:2]` starts at index `1` (value `2`), stops before index `5`, and steps by `2`, picking elements at index `1` and `3`, producing `[2, 4]`.

Q11. Correct Answer: B

Tuples can store references to any object, including mutable objects like lists. While the tuple reference itself cannot change, the internal contents of the nested list can be mutated freely.

Q12. Correct Answer: C

When called without parameters, `pop()` removes and returns the last item in the list.

Q13. Correct Answer: C

The `+` operator on lists performs sequence concatenation, joining elements into a single flat list: `[1, 2, 3, 4, 5, 6]`.

Q14. Correct Answer: C

`zip()` takes iterables, aggregates corresponding elements from each, and returns an iterator of tuples.

Q15. Correct Answer: C

Checking for an element's membership in a standard Python list using `in` requires scanning items sequentially, resulting in linear time complexity O(n).

Q16. Correct Answer: B

`tup[2]` retrieves the sublist `[3, 4]`. Calling `.append(5)` mutates that list directly in place, resulting in `(1, 2, [3, 4, 5])` without violating tuple immutability.

Q17. Correct Answer: A

The `count(value)` method returns the number of times `value` appears in the tuple.

Q18. Correct Answer: C

`insert(index, element)` places the element at the given index and shifts existing elements rightward. `nums.insert(1, 15)` puts `15` at index `1`, resulting in `[10, 15, 20, 30]`.

Q19. Correct Answer: B

Tuple unpacking allows Python to assign individual elements of a tuple directly to a comma-separated tuple of variables in a single operation.

Q20. Correct Answer: B

Default argument values in Python are evaluated once when the function is defined, not on each call. The default list `list_name` persists across function invocations, accumulating `1` then `2`.

Q21. Correct Answer: C

Item assignment on a tuple (e.g., `tup[1] = 99`) raises a `TypeError: 'tuple' object does not support item assignment` because tuples are immutable.

Q22. Correct Answer: A

Slice assignment replaces the specified sub-range (`lst[1:3]`, which covers indices 1 and 2) with the items of the assigned iterable, expanding or contracting the list as needed. Thus, `[2, 3]` is replaced by `[9, 9, 9]`, giving `[1, 9, 9, 9, 4]`.

Q23. Correct Answer: B

Dictionary keys and set elements must be hashable. Because tuples are immutable, they implement a fixed hash value and can be used as keys, whereas mutable lists cannot.

Q24. Correct Answer: A

`b = a[:]` creates a shallow copy of list `a`. Appending `4` to `b` does not affect `a`, which remains `[1, 2, 3]`.

Q25. Correct Answer: B

`remove(value)` searches for the first item in the list matching `value` and deletes it. If the value is not found, it raises a `ValueError`.

Q26. Correct Answer: B

Slicing with a negative step value `[::-1]` reverses any sequence in Python, including tuples. The output is `(5, 4, 3, 2, 1)`.

Q27. Correct Answer: B

The `clear()` method removes all items from an existing list in place, resetting its length to `0` (leaving `[]`).

Q28. Correct Answer: A

Extended iterable unpacking assigns `a = 1`, `c = 5`, and the starred variable `*b` collects all middle elements into a list: `[2, 3, 4]`.

Q29. Correct Answer: B

Under the hood in CPython, Python lists are implemented as dynamic arrays (contiguous arrays of pointers that resize dynamically as elements are added/removed).

Q30. Correct Answer: B

Passing a list iterable to the `tuple()` constructor converts the elements of the list into a new tuple object: `(1, 2, 3)`.